AI ASSISTED CODING ASS-16.1

2303A51678

B-28

Task 1 – Student Information System Schema

Task: Design a database schema for a Student Information System generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

- Insert a new student record.
- Fetch all courses enrolled by a student.
- Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student–course relationships.

CODE:

```python
import sqlite3

conn = sqlite3.connect("student_system.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    age INTEGER
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    credits INTEGER
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    enrollment_date TEXT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
)
""")
```

```python
conn.commit()

cursor.execute("INSERT INTO Students (name, email, age) VALUES (?, ?, ?)",
               ("Harsha", "harsha@example.com", 20))

cursor.executemany("INSERT INTO Courses (course_name, credits) VALUES (?, ?)", [
    ("DBMS", 4),
    ("OS", 3),
    ("AI", 4)
])

conn.commit()

cursor.executemany("INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (?, ?, ?)", [
    (1, 1, "2026-03-25"),
    (1, 2, "2026-03-25")
])

conn.commit()

print("Courses enrolled by student 1:")
cursor.execute("""
SELECT c.course_id, c.course_name, c.credits
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.student_id = ?
""", (1,))

for row in cursor.fetchall():
    print(row)
```

```python
print("\nStudent count in each course:")
cursor.execute("""
SELECT c.course_name, COUNT(e.student_id)
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id
""")

for row in cursor.fetchall():
    print(row)

conn.close()
```

Output:

```
Courses enrolled by student 1:
(1, 'DBMS', 4)
(2, 'OS', 3)

Student count in each course:
('DBMS', 1)
('OS', 1)
('AI', 0)
```

Task 2 Hospital Management Database

Task: Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

CODE:

```
import sqlite3

conn = sqlite3.connect("hospital.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    specialization TEXT,
    email TEXT UNIQUE
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    age INTEGER CHECK(age > 0),
    gender TEXT,
    phone TEXT UNIQUE
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER,
    patient_id INTEGER,
    appointment_date TEXT NOT NULL,
    status TEXT DEFAULT 'Scheduled',
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
)
""")
```

```python
print("Appointments for Doctor 1:")
cursor.execute("""
SELECT a.appointment_id, p.name, a.appointment_date, a.status
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = ?
""", (1,))
for row in cursor.fetchall():
    print(row)

print("\nPatient History (Patient 1):")
cursor.execute("""
SELECT d.name, a.appointment_date, a.status
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = ?
ORDER BY a.appointment_date DESC
""", (1,))
for row in cursor.fetchall():
    print(row)

print("\nTotal Patients per Doctor:")
cursor.execute("""
SELECT d.name, COUNT(DISTINCT a.patient_id)
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id
""")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Output:

```
Appointments for Doctor 1:
(1, 'Harsha', '2026-03-25', 'Scheduled')
op/AI Assitant Coding/hospital.py"
Appointments for Doctor 1:
(1, 'Harsha', '2026-03-25', 'Scheduled')
(2, 'Anita', '2026-03-26', 'Scheduled')
Appointments for Doctor 1:
(1, 'Harsha', '2026-03-25', 'Scheduled')
(2, 'Anita', '2026-03-26', 'Scheduled')
(2, 'Anita', '2026-03-26', 'Scheduled')

Patient History (Patient 1):

Patient History (Patient 1):
('Dr. Mehta', '2026-03-27', 'Scheduled')
('Dr. Mehta', '2026-03-27', 'Scheduled')
('Dr. Rao', '2026-03-25', 'Scheduled')
('Dr. Rao', '2026-03-25', 'Scheduled')

Total Patients per Doctor:
('Dr. Rao', 2)
('Dr. Rao', 2)
('Dr. Mehta', 1)
```

Task 3 Library Database

Task: Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

CODE:

```python
import sqlite3

conn = sqlite3.connect("library.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT,
    available INTEGER DEFAULT 1
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER,
    member_id INTEGER,
    loan_date TEXT NOT NULL,
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
)
""")

conn.commit()
```

```python
print("Currently Issued Books:")
cursor.execute("""
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
""")
for row in cursor.fetchall():
    print(row)

print("\nOverdue Books (>30 days):")
cursor.execute("""
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < DATE('now', '-30 days')
""")
for row in cursor.fetchall():
    print(row)

print("\nBooks per Member:")
cursor.execute("""
SELECT m.name, COUNT(l.loan_id)
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id
""")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Output:

```
Currently Issued Books:
('Clean Code', 'Harsha', '2026-02-10')
op/AI Assitant Coding/LIBRARY.py"
Currently Issued Books:
('Clean Code', 'Harsha', '2026-02-10')
('Clean Code', 'Harsha', '2026-02-10')
('DBMS Guide', 'Anita', '2026-03-20')

Overdue Books (>30 days):
('Clean Code', 'Harsha', '2026-02-10')

Overdue Books (>30 days):
('Clean Code', 'Harsha', '2026-02-10')
('Clean Code', 'Harsha', '2026-02-10')

Books per Member:
('Harsha', 2)
('Anita', 1)
Books per Member:
('Harsha', 2)
('Anita', 1)
('Harsha', 2)
('Anita', 1)
('Anita', 1)
```

Task 4 Real-Time Application: Online Shopping Database

Scenario: Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

CODE:

```
import sqlite3

conn = sqlite3.connect("ecommerce.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    price REAL NOT NULL,
    stock INTEGER DEFAULT 0
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    order_date TEXT NOT NULL,
    total_amount REAL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
)
""")
```

–

```python
print("Orders by User 1:")
cursor.execute("""
SELECT order_id, order_date, total_amount
FROM Orders
WHERE user_id = ?
""", (1,))
for row in cursor.fetchall():
    print(row)

print("\nMost Purchased Product:")
cursor.execute("""
SELECT p.name, SUM(od.quantity) as total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_sold DESC
LIMIT 1
""")
print(cursor.fetchone())

print("\nTotal Revenue (March 2026):")
cursor.execute("""
SELECT SUM(total_amount)
FROM Orders
WHERE strftime('%m', order_date) = '03'
AND strftime('%Y', order_date) = '2026'
""")
print(cursor.fetchone())

conn.close()
```

Output:

```
Orders by User 1:
(1, '2026-03-10', 52000.0)
(2, '2026-03-15', 20000.0)

Most Purchased Product:
('Headphones', 2)

Total Revenue (March 2026):
(94000.0,)
```

Task 5 University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrityconstraints.

• Generate queries:

- Insert examination results for a student.
- Retrieve subject-wise marks for a student. □ Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

CODE:

```
import sqlite3

conn = sqlite3.connect("university.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Subjects (
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_name TEXT NOT NULL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_id INTEGER,
    exam_date TEXT NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    exam_id INTEGER,
    marks INTEGER CHECK(marks >= 0 AND marks <= 100),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
)
""")
conn.commit()
```

```python
print("Subject-wise Marks:")
cursor.execute("""
SELECT s.subject_name, r.marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = ?
""", (1,))
for row in cursor.fetchall():
    print(row)

print("\nAverage Marks per Subject:")
cursor.execute("""
SELECT s.subject_name, AVG(r.marks)
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id
""")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Output:

```
Subject-wise Marks:
('DBMS', 85)
('OS', 90)
('AI', 88)

('AI', 88)

Average Marks per Subject:
('DBMS', 85.0)
('DBMS', 85.0)
('OS', 90.0)
('AI', 88.0)
```